UNIT-II

Syntax I: Parsing Natural Language, Treebanks: A Data-Driven Approach to Syntax, Representation of Syntactic Structure, Parsing Algorithms.

Introduction:

Parsing uncovers the hidden structure of linguistic input. In many applications that involve natural language, the underlying predicate-argument structure of sentences can be useful.

The syntactic analysis of language provides a means to explicitly discover the various predicate argument dependencies that may exist in a sentence.

In NLP, the syntactic analysis of natural language input can vary from being very low-level, such as simply tagging each word in the sentence with a part of speech (POS), or very high level, such as recovering a structural analysis that identifies the dependency between each predicate in the sentence and its explicit and implicit arguments.

In syntactic parsing, ambiguity is a particularly difficult problem because the most plausible analysis must be chosen from an exponentially large number of alternative analyses.

From tagging to full parsing, algorithms that can handle such ambiguity have to be carefully chosen.

Here we explore the syntactic analysis methods from tagging to full parsing and the use of supervised machine learning to deal with ambiguity.

Q) Explain the problems by using context free grammar for syntactic analysis of natural language.

Parsing of a natural language is a process of determining syntactic structure of the text by analyzing the words based on underlying grammar.

It identifies the information which is not explicitly given in the input sentence.

For doing this task it requires some additional information, that is grammar of language.

Context free grammar (CFG) is used to write the rules of syntax

Typically, the rules are stated in the form X → w; where X is a part of speech for word w which is a generated terminal symbol.

For example: in rule VB → saw, VB is a part of speech or preterminal symbol which generates verb saw.

for example: here we have the sentence "**john bought a shirt with pockets**".

Context free grammar (CFG):

S → NP VP

NP → DT NN | NN PP

VP → VB NP

VB → bought

DT → a

NN → john | shirt | pockets

PRP → with

Where VB = Verb, NP= Noun Phrase, VP = Verb Phrase, DT= Determiner,

PP = Prepositional Phrase, PRP= Preposition.

The sentence john bought a shirt with pockets can have two possible forms based on rules stated in CFG.

In the first parse form pockets can be considered as a currency to buy a shirt and in second form the sentence can be of meaning john is purchasing shirt with pockets.

1st Parse : (S (NP John)
(VP (VP (VB bought)
(NP (DT a)
(N shirt)))
(PP(PRP with)
(NP Pockets))))

2nd Parse: (S (NP John)
(VP (VB bought)
(NP (NP (DT a)
(NN shirt))
(PP (PRP with)
(NP pockets)))))

From the above example we understand that writing all possible syntactic formations in a language is difficult and complex task as we cannot list down a single form. All the possibilities need to explore based on the part of speech tags mentioned for a particular word.

It is also difficult to mention lexical properties of a particular word, which is a first knowledge acquisition problem which is a process of extracting structuring and organizing knowledge.

Second, problem of knowledge acquisition arises from the fact that, it is not only sufficient to know the syntactic rules of a language, but it is also required to understand which analysis is most feasible for the sentence in terms of meaning due to ambiguity in syntactic analysis.

For example: Consider CFG

N → NN

N → 'natural' | 'language' | 'processing' |'book'

If we consider Input sentence as natural language processing two possible ambiguous parses can be generated.

Parse 1:(N (N(N natural)
(N language))
(N processing))

Parse 2: (N (N natural)
(N (N language)
(N processing)))

Parsi 1 leads to the meaning processing of natural language and Parsi 2 leads to meaning natural way to language processing.

This ambiguity problem is a limitation of CFG.

Parsing Natural Language:

Q) What are the motivations behind the Parsing Natural Language?

In a text-to-speech application, Input sentences are to be converted to a spoken output that should sound like it was spoken by a native speaker of the language**.**

The motivations behind the parsing natural language are:

- Natural Intonation
- Summarization
- Paraphrasing of Text

Natural Intonation:

Parsing can provide structural description That identifies a break in the sentences.

Consider the following pair of sentences.

1. He wanted to go for a drive in movie.
2. He wanted to go for a drive in the country.

There is a Natural pause between the words in sentence-2 That reflects an underlying hidden structure to the sentence.

A simple case occurs in the following sentence.

3. The cat who lives dangerously had nine lives.

In this case, a text-to-speech system needs to know that the first instance of the word lives is a verb, and the second instance is a noun before it can begin to produce the natural intonation for this sentence.

This is an instance of the parts of speech tagging problem where each word in the sentence is assigned a most likely part-of-speech.

Summarization:

Another motivation for parsing comes from the natural language task of summarization, in which several documents about the same topic should be condensed down to a small digest of information Typically limited in size to 100 or 250 words. Sach a summary may be in response to a question that is answered in a set of documents.

In this case, a useful **subtask** is to **compress an individual sentence** so that only the relevant portion of a sentence is included in the summary. This allows the summary to be concise, informative and fluent.

We may want to compress sentence 4 to a shorter sentence 5.

4. Beyond the basic level, the operations of the three products vary widely.
5. The operations of the products vary.

The elegant way to approach this task is to first parse the sentence to find the various constituents where we recursively partition the words in the sentence into individual phrases such as a verb phrase or a noun phrase.

The parse tree produced by the parser can now be edited using a compression model that is aware of constituents and a few choice constituent deletions can produce a fluent compressed version of the original sentence.

Paraphrasing of Text:

paraphrasing replaces the words or phrases with other words or phrases without changing the meaning of the sentence.

In sentence fragment 6 the capitalized phrase EUROPEAN COUNTRIES can be replaced with the other Phrases without changing the essential meaning of this sentence.

A few examples of replacement phrases are shown in italics in sentence fragments 7 to 11.

6. Open borders imply increasing racial fragmentation in EUROPEAN COUNTRIES.
7. Open borders imply increasing racial fragmentation in *the countries of Europe*
8. Open borders imply increasing racial fragmentation in *European states*
9. Open borders imply increasing racial fragmentation in *Europe*
10. Open borders imply increasing racial fragmentation in *European nations*
11. Open borders imply increasing racial fragmentation in *European countries*

This kind of replacement cannot simply rely on substitution of arbitrary words in the sentence because such an approach can led to incoherent and disfluent paraphrases.

In contemporary NLP, syntactic parsers are routinely used in many applications, including but not limited to statistical machine translation, information extraction from text collections, language summarizations producing entity grinds for language generation, error correction in text.

Treebanks: A Data-Driven Approach to Syntax:

Q) How to knowledge acquisition problems of context free grammar is addressed by treebank approach.

OR

Write note on Treebank approach of syntax analysis.

Due to ambiguous nature of a language two knowledge acquisition problems arise in syntactic analysis.

In natural language, it is far too complex to simply list all the syntactic rules in terms of a CFG.

The second knowledge acquisition problem not only do we need to know the syntactic rules for a particular language, but we also need to know which analysis is the most plausible for a given input sentence.

To address these problems the construction of treebank is a data driven approach to syntax analysis that allows us to address both knowledge acquisition bottlenecks in one stroke.

A **treebank** is simply a collection of sentences (also called a corpus of text), where each sentence is provided a complete syntax analysis. The syntactic analysis of each sentence has been judged by a human expert as the most plausible analysis for the sentence.

Treebank contains annotations of syntactic structure for large set of sentences.

Supervised machine learning techniques are used to train the parser from the training data extracted from treebanks.

The first knowledge acquisition problem is addressed by providing syntactic analysis directly instead of grammar.

The second knowledge acquisition problem is solved as for each sentence in a tree bank the most feasible syntactic analysis is provided.

Using supervised machine learning techniques are used to learn scoring function for all possible syntax analysis.

Representation of Syntactic Structure:

In Natural Language Processing (NLP), the syntactic structure of a sentence refers to its grammatical arrangement, which is crucial for understanding the relationships between words and phrases. This structure is typically represented by using Dependency Graphs to connect a word- the head of a phrase -with the dependents in that phrase. The notion connects a head with its dependents using a directed

connection. And parse trees or syntax trees, which visually depict the hierarchical organization of a sentence.

Syntax Analysis Using Dependency Graphs:

Q) how syntax analysis is done using Dependency Graphs.

The main philosophy behind dependency graphs is to connect a word - the head of a phrase with the dependents in that phrase. The notation connects a head with its dependent using a directed connection.

Dependency graphs, just like phrase structures trees, are a representation that is consistent with many different linguistic frameworks. The head-dependent relationship could be either semantic(head-modifier) or syntactic(head-specifier).

The words in the input sentence are treated as the only vertices in the graph, which are linked together by directed arcs representing syntactic dependencies.

In dependency based syntactic parsing, the task is to derive a syntactic structure for an input sentence by identifying the syntactic head of each word in the sentence.

This defines a dependency graph, where the nodes are the words of the input sentence and arcs are the binary relations from head to dependent.

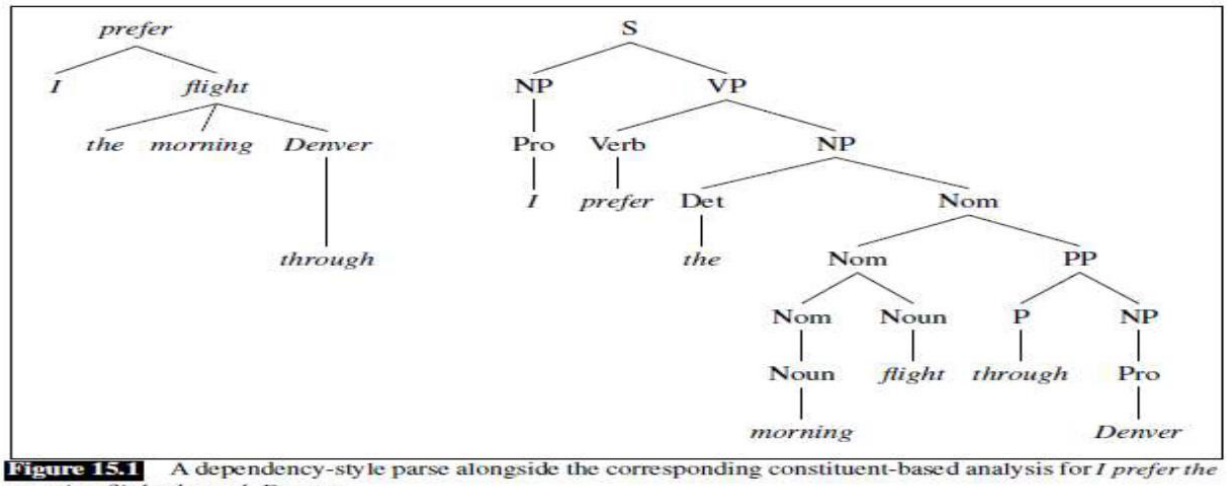
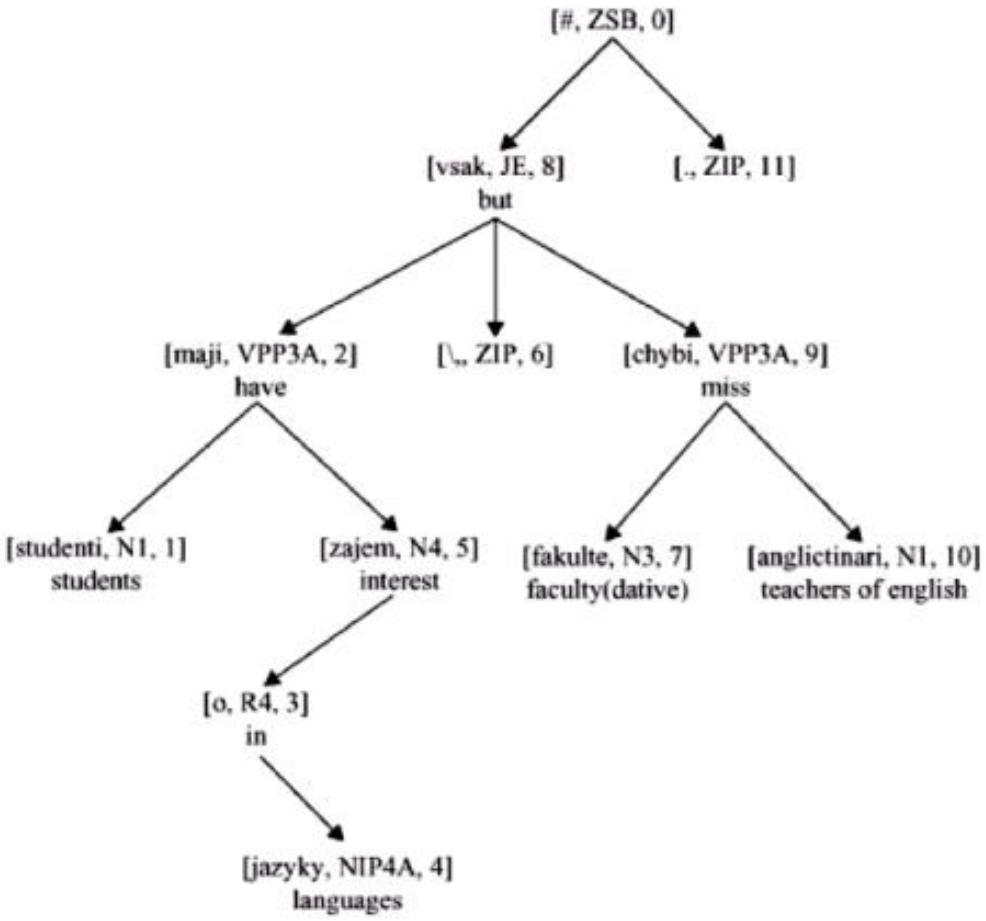


Figure 15.1 A dependency-style parse alongside the corresponding constituent-based analysis for *I prefer the morning flight through Denver.*

The dependency tree analyses where each word depends on exactly one parent, either another word or a dummy root symbol.

By convention, in dependency trees the 0 index is used to indicate the root symbol and the directed arcs are drawn from the head word to the dependent word.

The following figure shows an example of a dependency tree for a Czech sentence taken from the Prague dependency treebank, which is large corpus of Czech text annotated with dependency tree.



The students are interested in languages, but the faculty is missing teachers of English.

An example of a dependency graph syntax analysis for a Czech sentence taken from the Prague Dependency Treebank. Each node in the graph is a word, its parts of speech, and the position of the word in the sentence: for example [fakulte, N3,7] is the seventh word in the sentence with POS tag N3. Which also tells us that the word has dative case. The node [#, ZSB ,0] is the root node of the dependency tree. The English equivalent is provided for each node,

It is observed that dependency analysis makes minimal assumption about syntactic structure for avoiding annotation of hidden structure like empty elements to represent missing arguments of predicates.

Q) what is projective dependency tree?

Projectivity is the constraint on syntactic analysis due to effect of linear order of words on dependencies between words. A projective dependency tree is one where if we put the words in a linear order based on the sentence with the root symbol in the first position the dependency arcs can be drawn above the words without any crossing dependencies.

Another way to state projectivity is to say that for each word in the sentence, it's dependents f0rm a contiguous substring of the sentence.



An unlabeled nonprojective dependency tree with a crossing dependency

For example, the above figure shows a natural analysis of an English sentence that contains an extraposition to the right of a noun phrase modifier phrase, which as a result requires a crossing dependency.

Q) Explain syntax analysis using phrase structure tree?

A phrase structure syntax analysis of a sentence derives from the traditional sentence diagrams that partition a sentence into constituents, and larger constituents are formed by merging smaller ones.

phrase structure analysis also typically incorporate ideas from generative grammar to deal with displaced constituents or apparent long-distance relationships between heads and constituents.

A phrase structure tree can be viewed implicitly having a predicate argument structure associated with it.

For example, the following phrase structure analysis of the sentence "**Mr. baker seems especially sensitive"**, taken from the penn tree bank.

Phrase structure tree:

(S (NP-SBJ (NNP Mr.)

(NNP Baker))

(VP (VBZ seems)

(ADJP – PRD (RB especially)

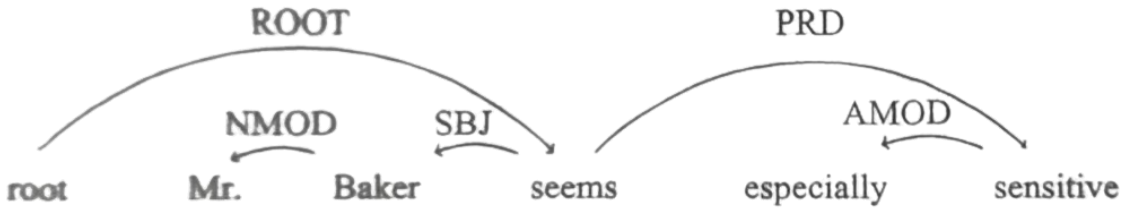
(JJ sensitive))))

The above phrase structure tree shows that the subject of the sentence is marked with the -SBJ marker and the predicate of the sentence is marked with the -PRD marker.

Predicate-argument structure:

Seems ((especially (sensitive)) (Mr. Baker))

Dependency tree can be drawn as:



Q) Explain about Parsing Algorithms?

In NLP, parsing is the process of analyzing a sentence to determine its grammatical structure and identify the relationships between its components. It's like breaking down a sentence into its building blocks to understand how the words fit together to convey meaning. This analysis helps computers to understand the structure and meaning of human language.

The core idea of passing decomposes a sentence into its constituent parts such as noun phrases, verb phrases and other grammatical units. These units are then organized into a hierarchical structure often represented as a tree.

A parsing algorithm, also known as a parser, is a method used in computer science to analyze a sequence of tokens to determine its grammatical structure according to formal grammar. The output of your parsing algorithm is typically a parse tree. which visually represents how the words fit together based on the rules of the language syntax.

There are several algorithms used in natural language processing for syntax analysis or parsing each with its own strengths and weaknesses.

There are three types of passing algorithms:

- Shift-Reduce Parsing
- Hypergraphs and Chart Parsing
- Minimum Spanning Trees and Dependency Parsing

Q) Explain the working of Shift-Reduce Parsing?

To build a parser, we need to create an algorithm that can perform the steps in the preceding rightmost derivation for any grammar and for any input string. CFG turns out to have an automaton that is equivalent to it, called a pushdown automaton.

A pushdown automaton is simply a finite-state automaton with some additional memory in the form of a stack. This is a limited amount of memory because only the top of the stack is used by the machine.

This Provides an algorithm for parsing that is general for any given CFG and input string. This algorithm is called **shift-reduce parsing**, which uses two data structures, a buffer for input symbols and a stack for storing CFG symbols, and is defined as follows:

1. Start with an empty stack and the buffer containing the input string.

2. Exit with success if the top of the stack contains the start of the grammar and if the buffer is empty.

3. Choose between the following two steps(if the choice is ambiguous, choose one based on an oracle):

   ➢ Shift a symbol from the input buffer onto the stack.

   ➢ If the top k symbols of the stack are α_1, α_2….. α_k which corresponds to the right-hand side of CFG rule A→ α_1, α_2….. α_k, then replace the top k symbols with the left-hand side nonterminal A.

4. Exit with failure if no action can be taken in previous step.

5. Else, go to step 2.

For the CFG G shown earlier in this section and for the input **a and b or c,** we show the individual steps in the Shift-reduce parsing in the following figure 3-4.

Context Free Grammar:

N → a

N → b

N → c

N → N or N

N → N and N

Parse Tree	Stack	Input	Action
		a and b or c	Init
a	a	and b or c	shift a
(N a)	N	and b or c	reduce N -> a
(N a) and	N and	b or c	shift and
(N a) and b	N and b	or c	shift b
(N a) and (N b)	N and N	or c	reduce N -> b
(N (N a) and (N b))	N	or c	reduce N -> a
(N (N a) and (N b)) or	N or	c	shift or
(N (N a) and (N b)) or c	N or c		shift c
(N (N a) and (N b)) or (N c)	N or N		reduce N -> c
(N (N (N a) and (N b)) or (N c))	N		reduce N -> N or N
(N (N (N a) and (N b)) or (N c))	N		Accept!

Figure 3–4: The individual steps of the shift-reduce parsing algorithm for the input *a and b or c* for the grammar *G* defined at the beginning of this section

Shift-Reduce Parsing Algorithm for Dependency Parsing:

The same algorithm can also be applied for dependency parsing, as can be seen by the example in following figure of using a shift-reduce parser for dependency parsing. At each step, the parser has a choice: either shit a new token into the stack or combine the top two elements of the stack with a head → dependent link or a dependent ← head link.

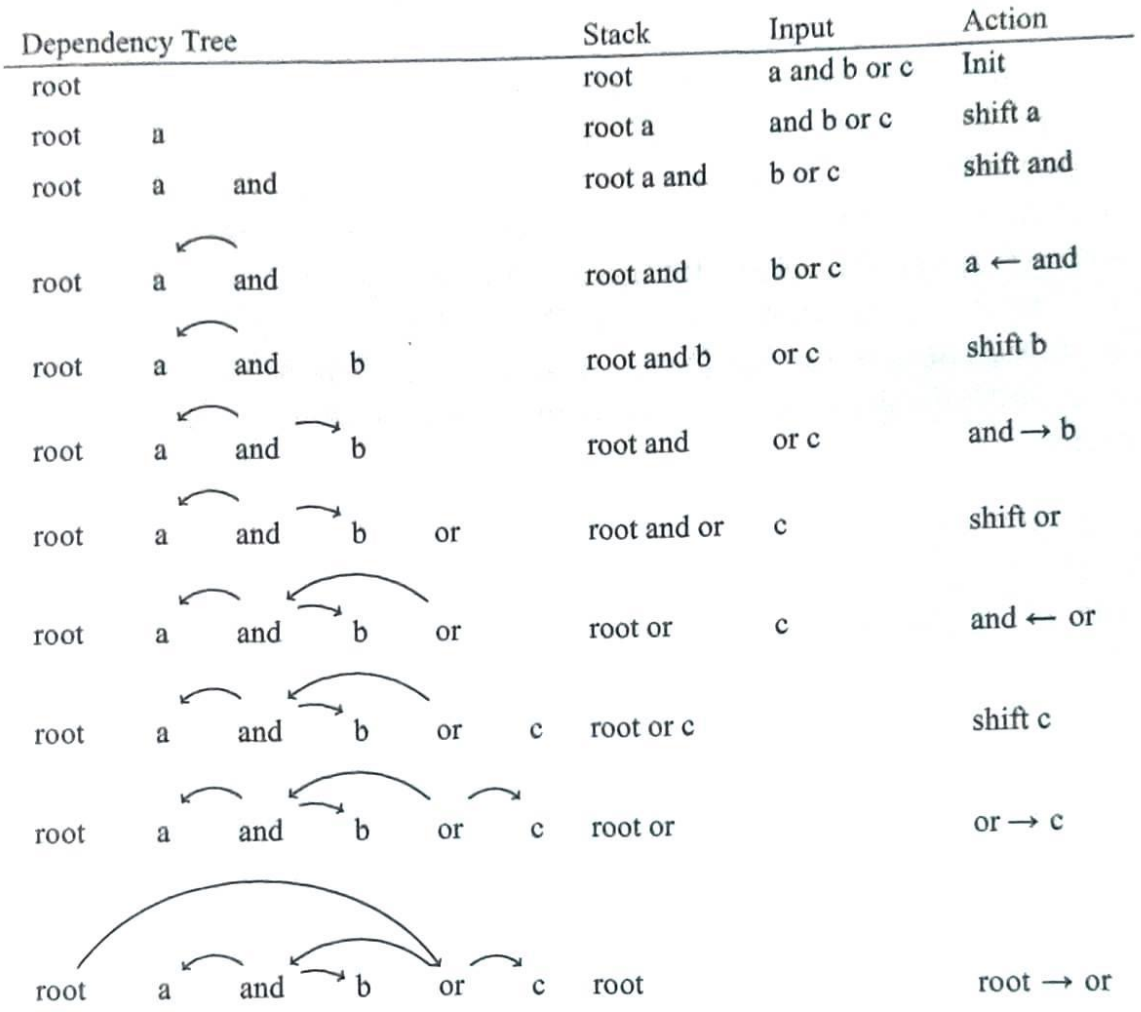


Figure 3–5: Steps of the shift-reduce parsing algorithm for dependency parsing

When using the shift-reduce algorithm in a statistical dependency parser, it helps to combine a shift action when possible.

Q) Explain hypergraphs and chart parsing?

Or

Explain CYK algorithm?

Shift-reduce parsing allows a linear time parse but requires access to an oracle. For general CFGs in the worst case, such a parser might have to resort to backtracking, which means reparsing the input, which leads to a time that is exponential in the grammar size in the worst case. On the other hand, CFGs do have a worst-case parsing algorithm that can run in $0(n^3)$ where n is the length of the input. Variants of this algorithm are often used in statistical parsers that attempt to search the space of possible parse trees without the limitation of purely left-to-right parsing.

Our example CFG G:

N → N 'and' N

N → N 'or' N

N →'a' | 'b' | 'c'

Is rewritten into a new CFG Gc where the right hand side only contains up to two nonterminals. This is done by introducing two new non terminals N^ and N_V

N → N N^

N^ → 'and' N

N → N N_V

N_V → 'or' N

N → 'a' | 'b' | 'c'

For example , for input string a and b or c, this new CFG Gf that represents the forest of parse trees is shown below. Imagine that the input string is broken up into

spans 0 a 1 and 2 b 3 or 4 c 5 so that a is span 0,1 and the string b or c is the span 2,5 in this string. The different parse trees that can be generated using this grammar are the valid parse trees for input sentence.

New Gf

$N[0,5] \rightarrow N[0,1]\ \hat{N}[1,5]$

$N[0,3] \rightarrow N[0,1]\ \hat{N}[1,3]$

$\hat{N}[1,3] \rightarrow$ 'and' $[1,2]\ N[2,3]$

$\hat{N}[1,5] \rightarrow$ 'and' $[1,2]\ N[2,5]$

$N[0,5] \rightarrow N[0,3]\ N_v[3,5]$

$N[2,5] \rightarrow N[2,3]\ N_v[3,5]$

$N_v[3,5] \rightarrow$ 'or' $[3,4]\ N[4,5]$

$N[0,1] \rightarrow$ 'a' $[0,1]$

$N[2,3] \rightarrow$ 'b' $[2,3]$

$N[4,5] \rightarrow$ 'c' $[4,5]$

- The parse tree that starts from a start nonterminal and spans complete string is shown in Fig. Q.7.1.

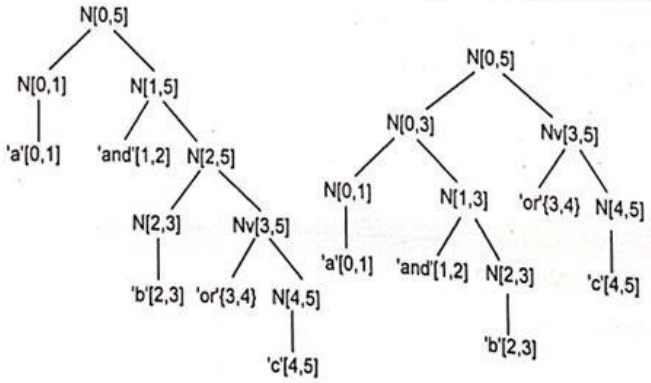


Fig. Q.7.1 : Parse trees embedded in the specialized CFG for a particular input string. The nodes with the same label, such as N[0,5], N[0,1] and [1,2], N[2,3] and Nv[3,5] can be merged to form a hypergraph representation of all parses for the input

- To construct this specialized CFG following are steps :
- First rules generating lexical items are considered

  For ex. N [0, 1] → 'a' [0, 1]

  N [2, 3] → 'b' [2, 3]

  N [4, 5] → 'c' [4, 5]
- The pseudorod is written as shown in Fig. Q.7.2.

```
for i = 0 ..... n do
    if N → x with score s for any x spanning i, i + 1 exists then
      add specialized rule N [i, i + 1] → x [i, i + 1] with score s
      written as N[i, i + 1] : s
    end if
end for
```

Fig. Q.7.2

- In the next step specialized rules are created recursively. For ex. If Y[i, k] and Z[k, j] are left hand sides of previously created rules then rule rate X → YZ can be converted to

  X [i, j] → Y [i, k] Z [k, j]

- Score S is assigned to each non-terminal span.
- The algorithm is shown in Fig. Q.7.3 and is known as CKY (Cocke, Kasami and younger) algorithm.

```
for j = 2 .... n do
  for i = j – 1 ... 0 do
    for k = i + 1 .... j do
      if Y [i, k] : s1 and Z[k, j] : s2 are in the specialized grammar then
        if X → YZ with score s exists in the original grammar then
          add specialized rule X[i, j] → Y[i, k] Z[k, j] with score s + s1 + s2
          keep only the highest scoring rule : X[i, j] → α
        end if
      end if
    end for
  end for
end for
```

Fig. Q.7.3

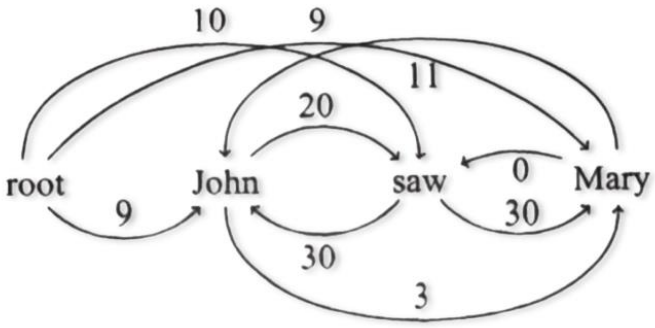
Q) Explain the minimum spanning tree for dependency parsing with an example.

Minimum spanning tree (MST): Minimum spanning tree (MST) algorithm are often used for dependency parsing, as they provide an efficient way to find the most likely parse for a sentence given a set of syntactic dependencies.

Dependence parsing: Dependence parsing is a type of syntactic parsing that represents the grammatical structure of a sentence as a directed acyclic graph (DAG). The nodes of the graph represented the words of the sentence, and the edges represent the syntactic relationship between the words.

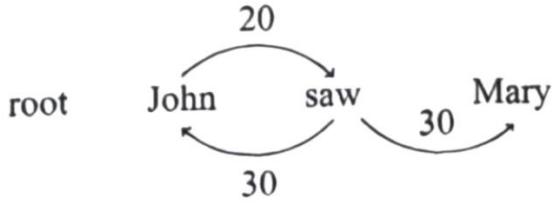
In NLP, the tradition is to use the term minimum spanning tree to refer to the optimum branching problem in directed graph.

Let us consider the following fully connected graph for the input sentence John saw Mary. The edges have weights based on some scoring function on edges.



The first step is to find the highest scoring incoming edge. If this step results in a tree, when we report this as the parse because it would have to be an MST.

In this example, however, after we pick only the highest scoring incoming edge from the graph, we do have a cycle.



We can contract the cycle into a single node, and we recalculate the edge weights. When we compute the edge weights from each node to that connected node, we also must keep track of which component of the merged node is the one with maximum weight.

For example, in the preceding graph we compute the incoming edge:

root→ **saw→ john** **: Wt.=40** with

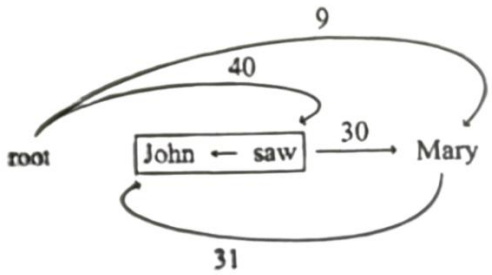
root→ **john→saw** **: Wt.=29** and

compare the incoming edge:

Mary → **saw → john** **: Wt.=30** with

Mary → **john → saw** **: Wt.=31**

To obtain the ones with maximum weight shown below:



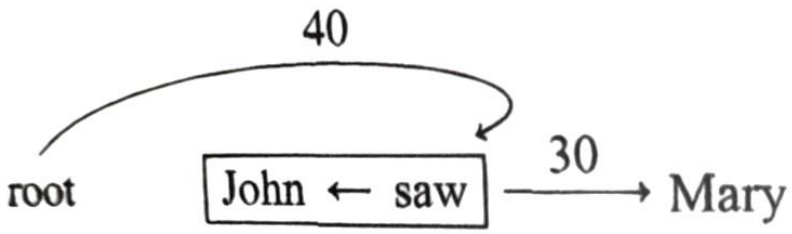
We now run the MST algorithm recursively on the graph, which means finding the graph with the best incoming edge to each word.

In this case it means comparing

root → Mary → **John-saw** **: Wt.= 9+31 with**

root → **John-saw** **→ Mary: Wt.= 40+30**

which results in the following graph.



Unwinding the recursive step provides us with the MST that is the highest scoring dependency parse of the input.

